9. Design a Scalable Web Crawler

Key Terms:

Term	Full Name	What it does
URL Frontier	URL Queue	Prioritized queue of URLs waiting to be crawled
robots.txt	Robots Exclusion	File telling crawlers which pages not to crawl
SimHash	Similarity Hash	Fingerprint for detecting near-duplicate content
MinHash	Min-wise Hashing	Algorithm for estimating similarity between sets
PageRank	Page Ranking	Algorithm scoring pages by incoming link importance
TF-IDF	Term Frequency-Inverse Document Frequency	Weights words by importance in document
BFS	Breadth-First Search	Crawl strategy visiting all pages at depth N before N+1
DNS	Domain Name System	Resolves domain names to IP addresses

Problem Statement

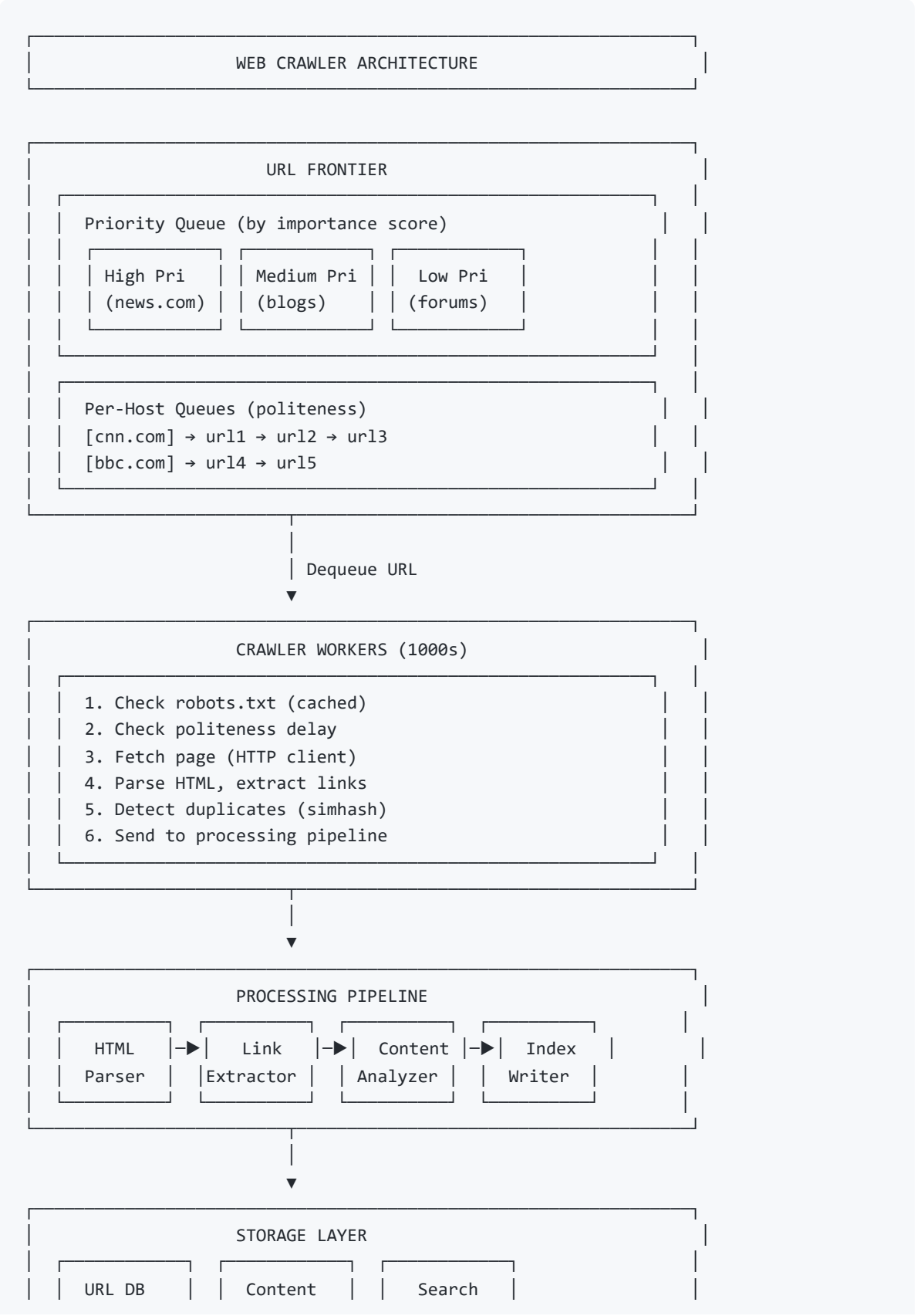
Design a web crawler like Bing's crawler that can index billions of web pages efficiently while being polite to websites.

Clarifying Questions to Ask

1. **Scale**: How many pages to crawl? (10B pages?)
2. **Freshness**: How often to recrawl? (Daily? Weekly?)
3. **Depth**: How deep to follow links? (5 levels?)
4. **Politeness**: Rate limits per domain? (1 req/sec?)
5. **Content types**: HTML only or PDFs, images too?

6. **Deduplication**: Handle duplicate/near-duplicate pages?
7. **Robots.txt**: Respect exclusions?

High-Level Design



```
│  │ (visited)  │  │   Store    │  │   Index    │               │
│  └────────────┘  └────────────┘  └────────────┘               │
└──────────────────────────────────────────────────────────────┘
```

API Design

Crawl Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/crawls`	Start a new crawl job
`GET`	`/api/v1/crawls/{crawlId}`	Get crawl job status
`POST`	`/api/v1/crawls/{crawlId}/stop`	Stop a running crawl
`GET`	`/api/v1/crawls/{crawlId}/results?page={n}`	Get crawled pages

Seed URL Management

Method	Endpoint	Description
`POST`	`/api/v1/seeds`	Add seed URLs
`GET`	`/api/v1/seeds`	List seed URLs
`DELETE`	`/api/v1/seeds/{seedId}`	Remove a seed URL

Configuration APIs

Method	Endpoint	Description
`PUT`	`/api/v1/config/politeness`	Set crawl rate limits per domain
`PUT`	`/api/v1/config/filters`	Set URL inclusion/exclusion rules
`GET`	`/api/v1/stats`	Get crawler statistics (pages/sec, queue depth)

Key Components

Component	Purpose	Technology
URL Frontier	Prioritized URL queue	Redis + Custom
Crawler Workers	Fetch web pages	Distributed workers

DNS Resolver	Resolve hostnames	Local cache + DNS
Robots Parser	Parse robots.txt	Custom parser
Duplicate Detector	Identify duplicates	SimHash / MinHash
Content Store	Store raw HTML	Blob storage
Link Extractor	Extract outbound links	HTML parser

URL Frontier Design

```
TWO-LEVEL QUEUE SYSTEM:

Level 1: Priority Queues (what to crawl)
┌─────────────────────────────────────────────────────────────┐
│  Priority based on:                                         │
│  - PageRank score                                           │
│  - Freshness (news sites crawled more often)                │
│  - Update frequency (changed pages prioritized)             │
└─────────────────────────────────────────────────────────────┘

Level 2: Per-Host Queues (politeness)
┌─────────────────────────────────────────────────────────────┐
│  Each host has its own queue                                │
│  Enforces delay between requests to same host               │
│  Respects robots.txt crawl-delay directive                  │
└─────────────────────────────────────────────────────────────┘

Scheduling:
  1. Select from highest priority queue
  2. Find a host that's not rate-limited
  3. Dequeue URL from that host's queue
  4. Update host's last-access timestamp
```

Duplicate Detection

```
SIMHASH (Near-duplicate detection):
┌─────────────────────────────────────────────────────────────┐
│  1. Extract features (word n-grams)                         │
│  2. Hash each feature                                       │
│  3. Weight by TF-IDF                                        │
│  4. Combine into 64-bit fingerprint                         │
│  5. Compare Hamming distance (< 3 = duplicate)              │
└─────────────────────────────────────────────────────────────┘

URL NORMALIZATION:
  http://Example.com:80/path/../page?b=2&a=1
  → https://example.com/page?a=1&b=2
```

```
- Lowercase hostname
- Remove default port
- Resolve path (../)
- Sort query parameters
- Remove tracking params (utm_*)
```

Scalability Considerations

- **Distributed crawling**: Partition URLs by hash(domain) to workers
- **DNS caching**: Cache DNS lookups (most lookups are repeats)
- **Connection reuse**: HTTP keep-alive to same hosts
- **Compression**: Request gzip, store compressed
- **Checkpointing**: Save frontier state for crash recovery
- **Adaptive crawling**: Crawl frequently-updated sites more often

Trade-offs

Decision	Option A	Option B	Recommendation
BFS vs DFS	BFS	DFS	**BFS** - better coverage
Politeness	Per-IP	Per-domain	**Per-domain** - safer
Storage	Store all	Store hashes	**Both** - full for index, hash for dedup
Scheduling	FIFO	Priority	**Priority** - important pages first

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	URL frontier queue and crawl task distribution	Tech Guide
Redis	URL deduplication and crawl state caching	Tech Guide
Bloom Filter	Efficient URL seen-before check	Tech Guide
PostgreSQL	Crawl metadata and robots.txt storage	Tech Guide

How to Explain It Well

> **Opening**: "A web crawler needs to be fast yet polite. I'll design for 10 billion pages with politeness guarantees and duplicate detection."

> **Key talking points**:
>
> - "Two-level frontier: priority queues for importance, per-host queues for politeness"
> - "SimHash fingerprints detect near-duplicates with Hamming distance < 3"
> - "Partition by domain hash so each worker owns a set of domains"
> - "Cache robots.txt and DNS to avoid repeated lookups"
> - "Adaptive recrawl: news sites hourly, static sites weekly"
